PEMROGRAMAN BERBASIS FRAMEWORK
JOBSHEET 17

NAMA : Muhammad Fahreza Rohmansyah
KELAS : 3E
NO. ABSEN : 15
NIM : 2341720151

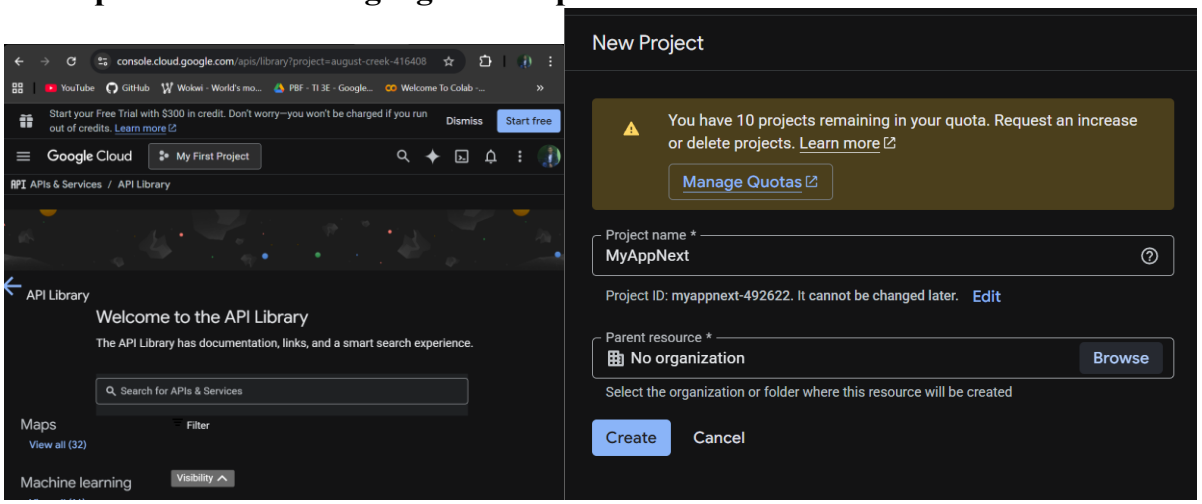
PROGRAM STUDI TEKNIK INFORMATIKA
JURUSAN TEKNOLOGI INFORMASI
POLITEKNIK NEGERI MALANG
2026

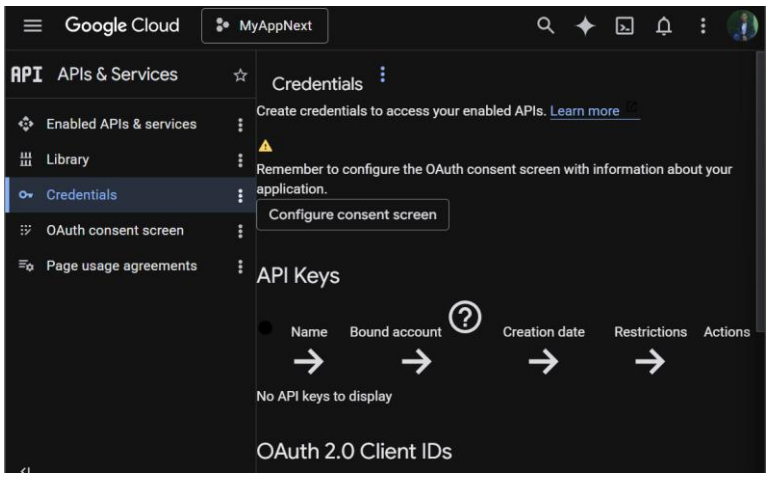
Link Github:

https://github.com/rezfahreza/PBF_2026.git

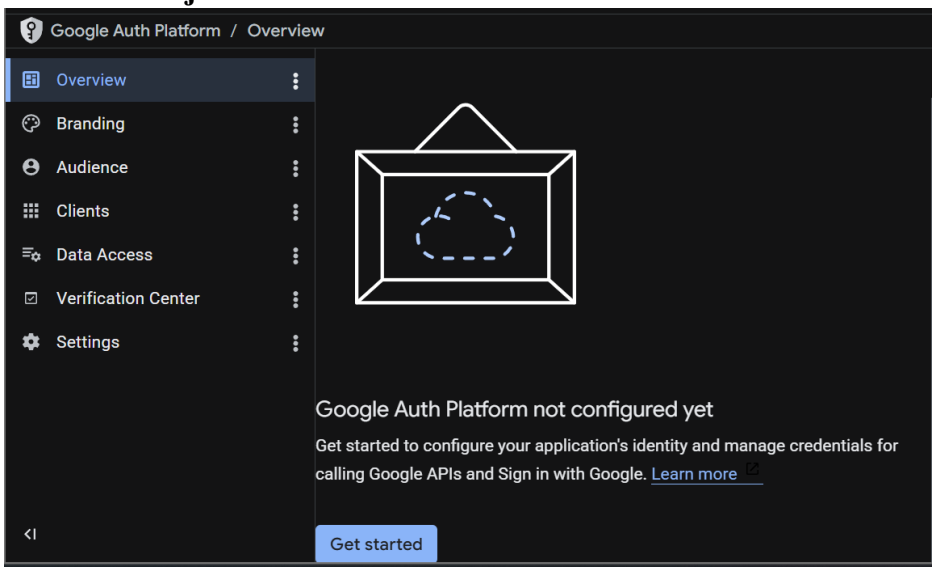
Praktikum

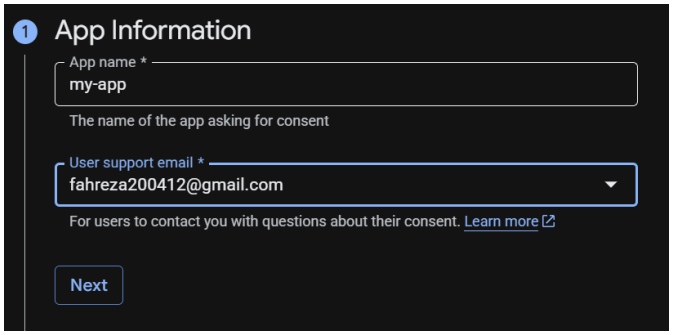
1. Masuk ke Google Cloud Console Buka: https://console.cloud.google.com/apis/credentials

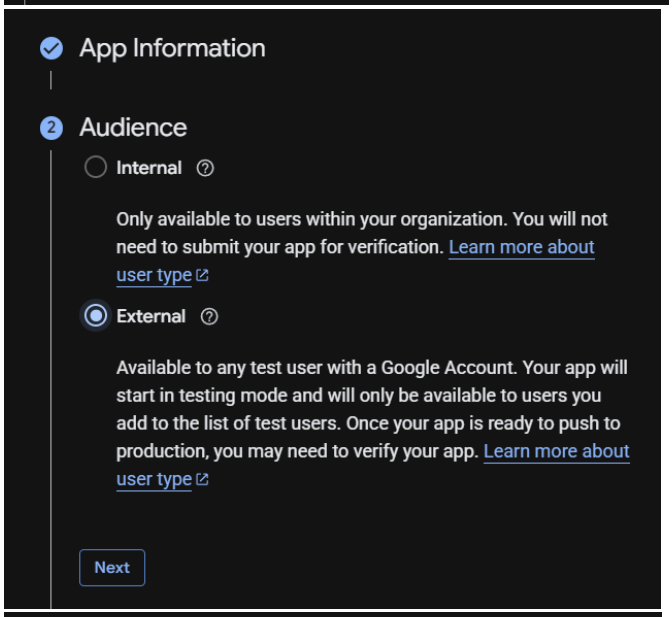


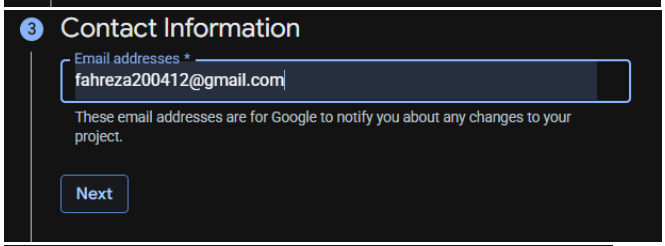


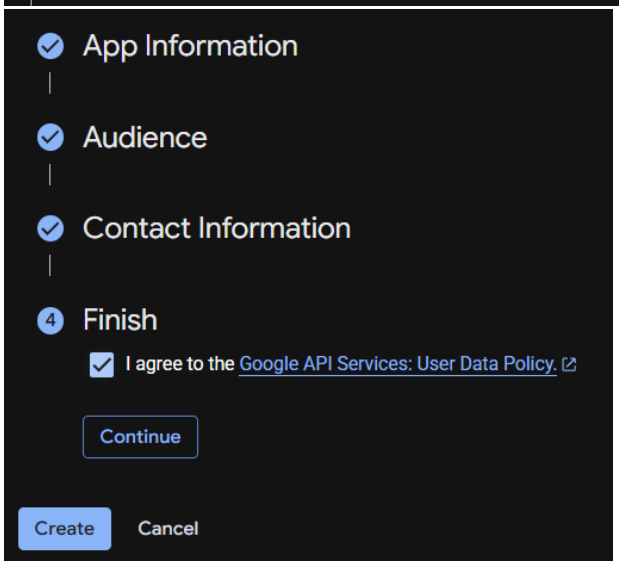
2. Buat Project Baru



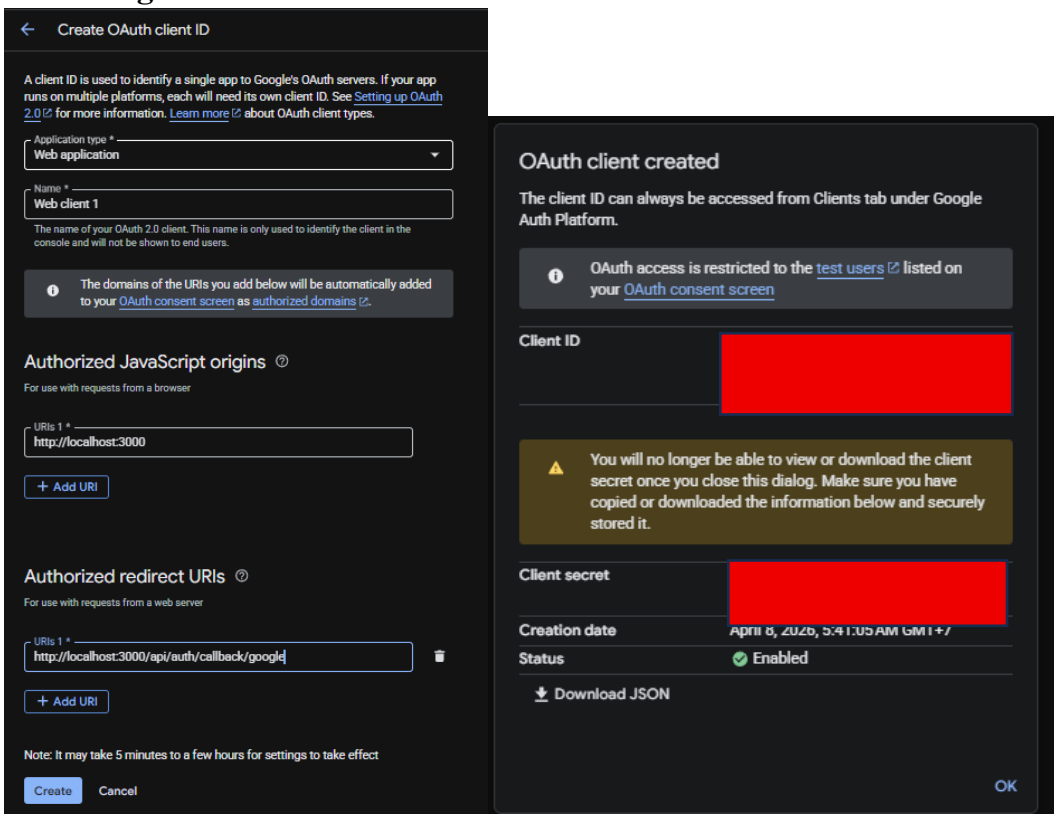

1 App Information
App name *
my-app
The name of the app asking for consent
User support email *
fahreza200412@gmail.com
For users to contact you with questions about their consent. Learn more
Next


App Information
2 Audience
Internal
Only available to users within your organization. You will not need to submit your app for verification. Learn more about user type
External
Available to any test user with a Google Account. Your app will start in testing mode and will only be available to users you add to the list of test users. Once your app is ready to push to production, you may need to verify your app. Learn more about user type
Next


3 Contact Information
Email addresses *
fahreza200412@gmail.com
These email addresses are for Google to notify you about any changes to your project.
Next


App Information
Audience
Contact Information
4 Finish
I agree to the Google API Services: User Data Policy.
Continue
Create
Cancel

3. Konfigurasi OAuth Consent Screen




```
GOOGLE_CLIENT_ID=[redacted]
GOOGLE_CLIENT_SECRET=[redacted]
```

4. Konfigurasi Google Provider di NextAuth dan Handle Callback JWT & Session

- Modifikasi [...nextauth].ts

```
7   export const authOptions: NextAuthOptions = {
8     session: {
9       strategy: "jwt"
10    },
11    secret: process.env.NEXTAUTH_SECRET,
12    providers: [
13      CredentialsProvider({ ···
44      }),
45      GoogleProvider({
46        clientId: process.env.GOOGLE_CLIENT_ID || "",
47        clientSecret: process.env.GOOGLE_CLIENT_SECRET || "",
48      }),
49    ],
```

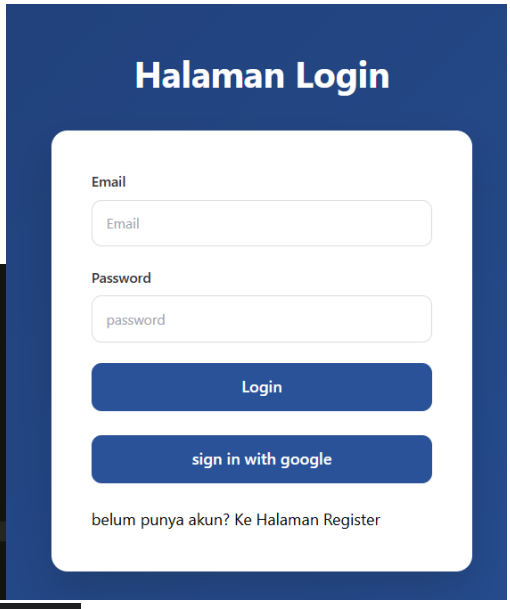
```
callbacks: {
  async jwt({ token, account, profile, user }:any) {
    if (account?.provider === "credentials" && user) {
      token.email = user.email;
      token.fullname = user.fullname;
      token.role = user.role;
    }
    if (account?.provider === "google") {
      const data = {
        email: profile?.email,
        fullname: profile?.name,
        image: profile?.picture,
        type: account.provider,
      };
      token.email = data.email;
      token.fullname = data.fullname;
      token.image = data.image;
      token.type = data.type;
    }
    return token
  },
```

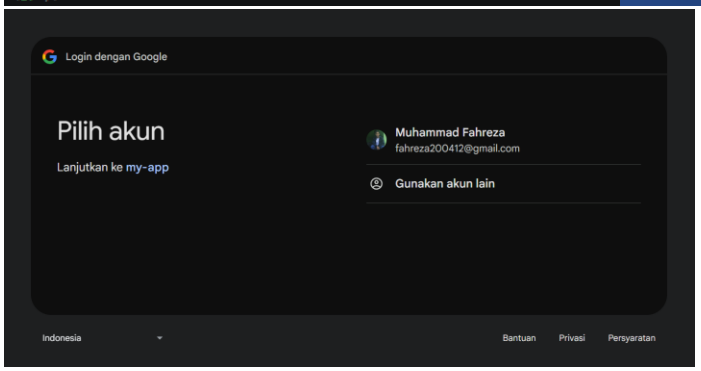
```
72    async session({ session, token }:any) {
73      if (token.email) {
74        session.user.email = token.email
75      }
76      if (token.fullname) {
77        session.user.name = token.fullname
78      }
79      if (token.role) {
80        session.user.role = token.role
81      }
82      if (token.image) {
83        session.user.image = token.image
84      }
85      if (token.type) {
86        session.user.type = token.type
87      }
88      return session
```

5. Tambahkan Button Login Google

- Modifikasi view/auth/login/index.tsx

```
    <button
        type="submit"
        className={style.login__form__item__button}
        disabled={isLoading}
    >
        {isLoading ? "Loading..." : "Login"}
    </button>
    <br /><br />
    <button
        onClick={() => signIn("google", {callbackUrl, redirect: false})}
        className={style.login__form__item__button}
        disabled={isLoading}
    >
        {isLoading ? "Loading..." : "sign in with google"}
    </button>
</form>
<br />
```





- Modifikasi Navbar untuk menampilkan image profile

```
    <div className={styles.navbar__user}>
        Welcome, {data.user.name}
        {data.user.image && (
            <Image
                src={data.user.image}
                alt={data.user.name || 'User avatar'}
                className={styles.navbar__user__image}
                width={42}
                height={42}
            />
        )}
    </div>
```

```
.navbar__user {
display: flex;
align-items: center;
gap: 12px;
font-size: 16px;
font-weight: 500;
opacity: 0.95;
}

/* Avatar image */
.navbar__user__image {
  width: 42px;
  height: 42px;
  border-radius: 50%;
  object-fit: cover;
  border: 2px solid #3b82f6;
  box-shadow: 0 0 10px rgba(59, 130, 246, 0.4);
  transition: all 0.3s ease;
}

.navbar__user__image:hover {
  transform: scale(1.1);
  box-shadow: 0 0 15px rgba(59, 130, 246, 0.8);
}
```

6. Simpan Data Google ke Database

- Modifikasi servicefirebase.ts

```
  query,
  where,
  updateDoc,
  addDoc,
} from "firebase/firestore";
import app from "./firebase";
import bcrypt from "bcrypt";

export type OAuthUserData = {
  email: string;
  fullname: string;
  image?: string;
  type: string; // e.g. "google" | "github"
  role?: string;
};

const db = getFirestore(app);

export async function retrieveProducts(collectionName: string) {...
}

export async function retrieveDataById(collectionName: string, id: string) {...
}

export async function signIn( email: string ) {...
}

export async function signUp(...
}

export async function upsertOAuthUser(userData: OAuthUserData) {
  try {
    const q = query(
      collection(db, "users"),
      where("email", "==", userData.email),
    );

    const querySnapshot = await getDocs(q);
    const data: any[] = querySnapshot.docs.map((doc) => ({
      id: doc.id,
      ...doc.data(),
    }));

    if (data.length > 0) {
      userData.role = data[0].role ?? "member";
      await updateDoc(doc(db, "users", data[0].id), userData);
      return {
        status: true,
        message: "User updated and logged in With google",
        data: { id: data[0].id, ...userData },
      };
    } else {
      userData.role = "member";
      const ref = await addDoc(collection(db, "users"), userData);
      return {
        status: true,
        message: "User registered and logged in With google",
        data: { id: ref.id, ...userData },
      };
    }
  } catch (error: any) {
    return {
      status: false,
      message: "Failed to register user With google",
    };
  }
}

// Backward-compatible wrapper used by existing auth code
export async function signInWithGoogle(userData: any, callback: any) {
  const result = await upsertOAuthUser(userData);
  callback(result);
}
```

- Panggil Service di JWT Callback buka file [...nextAuth].ts

```
  }
  if (account?.provider === "google") {
    const data = {
      email: profile?.email,
      fullname: profile?.name,
      image: profile?.picture,
      type: account.provider,
    };

    await signInWithGoogle(data, (result:any) => {
      if (result.status) {
        token.email = data.email;
        token.fullname = data.fullname;
        token.image = data.image;
        token.type = data.type;
      }
    });
  }
  return token
```

Tugas Praktikum

1. **Tambahkan role editor**
2. **Buat halaman khusus editor**
3. **Tambahkan provider GitHub**
4. **Refactor service agar reusable**
5. **Gunakan next/image untuk optimasi avatar**

Pertanyaan Refleksi

1. Apa perbedaan login credential dan login Google?

- Login Credential: User masuk menggunakan email dan password yang dibuat sendiri dan diverifikasi secara manual oleh sistem aplikasi kita.
- Login Google: Menggunakan akun Google yang sudah ada melalui protokol OAuth, sehingga user tidak perlu membuat password baru di aplikasi kita.

2. Mengapa data Google tetap perlu disimpan ke database?

Data Google tetap perlu disimpan agar sistem aplikasi kita dapat mengelola profil pengguna secara internal, seperti menentukan role (admin/user) , mencatat riwayat aktivitas, dan menyimpan data tambahan yang tidak disediakan oleh Google

3. Apa fungsi JWT callback?

Fungsi ini digunakan untuk memodifikasi atau memperkaya isi token (JWT) segera setelah user berhasil login. Contohnya adalah memasukkan data role atau fullname dari database ke dalam token agar informasi tersebut bisa langsung digunakan oleh aplikasi tanpa harus memanggil database berulang kali

4. Mengapa perlu multi-role?

Multi-role diperlukan untuk menerapkan Role-Based Access Control (RBAC). Hal ini memastikan adanya pembagian hak akses yang jelas, di mana fitur sensitif hanya bisa diakses oleh admin, sementara fitur umum dapat diakses oleh user biasa

5. Apa risiko jika tidak menyimpan user ke database?

- Kehilangan Kontrol: Kita tidak bisa mengelola status akun (misalnya memblokir user atau mengubah role) secara mandiri
- Keterbatasan Data: Aplikasi tidak bisa menyimpan informasi spesifik aplikasi yang terikat pada user tersebut, seperti alamat pengiriman atau poin loyalitas
- Masalah Keamanan: Sulit untuk melakukan validasi tambahan jika hanya mengandalkan penyedia pihak ketiga tanpa verifikasi internal.